

■ .NET 8 AI Migration Assistant Report

Project:	demo_project
Path:	C:\Users\Nandhini\Elevate8\demo_project
Generated at:	2026-06-09 17:37:27
Overall Risk Rating:	Low (14/100)
Migration Readiness:	86%
Estimated Effort:	1-3 Days (Low Effort)

1. Executive Summary

This migration report evaluates the structure, dependencies, and code configuration of the application located at C:\Users\Nandhini\Elevate8\demo_project. The assistant scanned legacy codebases, computed risk scores, and generated remediation strategies using Gemini models.

2. Migration Roadmap

- **Phase 1: Project Preparation** - Create full system backups and code checkpoints.
- **Phase 2: Target Upgrades** - Update build dependencies and change TARGET framework to net8.0.
- **Phase 3: Package Refactoring** - Upgrade legacy assemblies and NuGet dependencies.
- **Phase 4: API Remappings** - Code refactoring of WebForms/WCF bindings to Minimal APIs and gRPC.

3. Risk Blocker Summary

• **[Medium] System.Web**

Remediation: System.Web is deprecated. Replace HttpContext.Current usage with dependency injection of IHttpContextAccessor. Migrate handlers/modules to ASP.NET Core Middleware.

• **[Medium] Legacy Packages**

Remediation: Upgrade legacy NuGet packages to modern versions targeting .NET 8. Legacy packages found: EntityFramework, log4net, Microsoft.AspNet.Mvc, Microsoft.AspNet.Razor, Microsoft.AspNet.WebApi, Microsoft.AspNet.WebPages, Microsoft.AspNet.Web.Optimization, AutoFac.Mvc5, AjaxControlToolkit

• **[Low] Config Complexity**

Remediation: Convert XML configurations (Web.config/App.config) to JSON-based appsettings.json. Migrate WCF endpoint configurations to code-based initialization.

4. File-by-File AI Suggestions

File: Controllers\HomeController.cs

Summary: This file is an ASP.NET Core MVC controller that already leverages modern .NET 8 patterns, including constructor-based dependency injection for `IHttpContextAccessor`, `IMemoryCache`, `IConfiguration`, and `AppDbContext` (Entity Framework Core). It effectively replaces legacy `System.Web` APIs such as `HttpContext.Current.User`, `HttpContext.Current.Cache`, and `System.Web.HttpRequest`/`HttpResponse` cookie handling with their ASP.NET Core equivalents.

Strategy: The provided source file is already a well-migrated ASP.NET Core controller, meaning most of the heavy lifting for a .NET Framework to .NET 8 migration has already been completed. The following steps outline the conceptual transformation that *would have been* necessary, and which the provided code *already demonstrates*:

- Project File Conversion:** Convert the legacy .NET Framework `.csproj` to an SDK-style project format targeting `net8.0`.
- NuGet Package Update:** Replace `System.Web` specific packages (e.g., `Microsoft.AspNet.Mvc`, `EntityFramework`) with `Microsoft.AspNetCore.Mvc`, `Microsoft.EntityFrameworkCore`, `Microsoft.Extensions.Caching.Memory` and other ASP.NET Core equivalents.
- Controller Base Class:** Change `System.Web.Mvc.Controller` to `Microsoft.AspNetCore.Mvc.Controller`.
- HttpContext Access:** Replace `System.Web.HttpContext.Current` with constructor-injected `IHttpContextAccessor` for safe and testable access to HTTP context details (e.g., `User` identity).
- Caching Mechanism:** Migrate from `System.Web.Caching.Cache` (e.g., `HttpContext.Current.Cache`) to `Microsoft.Extensions.Caching.Memory.IMemoryCache`, injected via dependency injection.
- Configuration System:** Replace `System.Configuration.ConfigurationManager` with `Microsoft.Extensions.Configuration.IConfiguration` for application settings.
- Data Access Layer:** Ensure the data access uses Entity Framework Core (`Microsoft.EntityFrameworkCore.DbContext`) instead of legacy Entity Framework 6 or older ADO.NET patterns.
- Request/Response Objects:** Update usage of `System.Web.HttpRequest` and `System.Web.HttpResponse` for cookies and other properties to their `Microsoft.AspNetCore.Http` counterparts (e.g., `Request.Cookies`, `Response.Cookies.Append`, `CookieOptions`).
- Action Results:** Replace `System.Web.Mvc.ActionResult` with `Microsoft.AspNetCore.Mvc.IActionResult` and update specific action results (e.g., `HttpNotFound` to `NotFound()`, custom `Unauthorized` responses).
- Anti-Forgery Token:** Ensure `System.Web.Mvc.ValidateAntiForgeryTokenAttribute` is updated to `Microsoft.AspNetCore.Mvc.ValidateAntiForgeryTokenAttribute` and views correctly generate the token.

Migrated Equivalent Preview:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Http;
using Microsoft.Extensions.Caching.Memory;
using Microsoft.Extensions.Configuration;
using Microsoft.EntityFrameworkCore;

namespace LegacyApp.Controllers;

/// <summary>Home controller migrated to ASP.NET Core with dependency injection.</summary>
public class HomeController : Controller
{
    private readonly IHttpContextAccessor _httpContextAccessor;
    private readonly IMemoryCache _cache;
    private readonly IConfiguration _configuration;
    private readonly AppDbContext _db;

    /// <summary>
    /// Initializes a new instance of the <see cref="HomeController"/> class.
    /// </summary>
```

```
/// <param name="httpContextAccessor">The accessor for the current HTTP context.</param>  
/// <param name="cache">The in-mem  
///... [code truncated for length in PDF] ...
```